Covered are features like these:

- looking up translations
- interpolating data into translations
- pluralizing translations
- using safe HTML translations (view helper method only)
- localizing dates, numbers, currency, etc.

4.1 Looking up Translations

4.1.1 Basic Lookup, Scopes and Nested Keys

Translations are looked up by keys which can be both Symbols or Strings, so these calls are equivalent:

```
I18n.t :message
I18n.t 'message'
```

The `translate` method also takes a `:scope` option which can contain one or more additional keys that will be used to specify a "namespace" or scope for a translation key:

```
I18n.t :record_invalid, scope: [:activerecord, :errors, :messages]
```

This looks up the `:record_invalid` message in the Active Record error messages.
Additionally, both the key and scopes can be specified as dot-separated keys as in:

```
I18n.translate "activerecord.errors.messages.record_invalid"
```

Thus the following calls are equivalent:

```
I18n.t 'activerecord.errors.messages.record_invalid'
I18n.t 'errors.messages.record_invalid', scope: :activerecord
I18n.t :record_invalid, scope: 'activerecord.errors.messages'
I18n.t :record_invalid, scope: [:activerecord, :errors, :messages]
```

4.1.2 Defaults

When a `:default` option is given, its value will be returned if the translation is missing:

```
I18n.t :missing, default: 'Not here'
# => 'Not here'
```

If the `:default` value is a Symbol, it will be used as a key and translated. One can provide multiple values as default. The first one that results in a value will be returned.
E.g., the following first tries to translate the key `:missing` and then the key `:also_missing.` As both do not yield a result, the string "Not here" will be returned:

```
I18n.t :missing, default: [:also_missing, 'Not here']
# => 'Not here'
```

4.1.3 Bulk and Namespace Lookup

To look up multiple translations at once, an array of keys can be passed:

```
I18n.t [:odd, :even], scope: 'errors.messages'
# => ["must be odd", "must be even"]
```

Also, a key can translate to a (potentially nested) hash of grouped translations. E.g., one can receive*all* Active Record error messages as a Hash with:

```
I18n.t 'activerecord.errors.messages'
# => {:inclusion=>"is not included in the list", :exclusion=> ... }
```

4.1.4 "Lazy" Lookup